VAKRATRON SYSTEMS

WHITEPAPER · KUBERNETES AND PLATFORM ENGINEERING

Kubernetes and Platform Engineering: When, Why and How

Deciding whether you need Kubernetes, and building a platform your teams will actually use

Written for
CIOs, IT heads, platform and infrastructure architects, engineering managers

Version 1.0, October 2026
vakratronsys.com

Contents

01 Executive summary
02 Why Kubernetes platforms disappoint
03 What a platform actually is
04 The main options compared
05 Do you need Kubernetes? A fit test
06 Reference architecture
07 Sizing and cost
08 Security and governance
09 Operating it: day 2
10 A practical roadmap
11 Ten common mistakes
12 Platform readiness checklist (36 points)
13 Glossary
14 About Vakratron Systems

What you will take away

- An honest fit test, including when not to use Kubernetes at all
- Four ways to run containers compared on effort, cost and control
- What a platform includes beyond the cluster, and how to run it as a product
- Cluster design, tenancy and when a second cluster is justified
- A sizing formula and a worked cost example, people included
- The security and upgrade routine that keeps a platform safe to run
- A 36-point readiness checklist for new and existing platforms

01

Executive summary

Kubernetes is very good at one job: running many services for many teams, deploying them often and scaling them automatically. It is also complex. Most of the disappointment we see does not come from Kubernetes itself. It comes from adopting it for the wrong reasons, building more platform than anyone needed, and then finding that nobody has the time or skill to keep it upgraded.

This paper is for the people who decide whether to invest in a container platform and have to defend that decision a year later. It covers when Kubernetes is the right answer, what a platform needs beyond the cluster, and how to run one without it becoming a second job for everybody.

Five things to take away

1. **Start with an honest fit test.** A handful of applications and one team rarely need Kubernetes. Virtual machines with good automation, or a managed container service, are often simpler and cheaper.
2. **Build a platform, not just a cluster.** Developers need a working path from code to production: templates, delivery, security, monitoring and documentation. The cluster is perhaps a third of the work.
3. **Treat the platform as a product.** It has users, a roadmap and measures of success. If developers still raise tickets to get an environment, the platform has not done its job.
4. **Fewer clusters, run well.** One well-run production cluster beats five neglected ones. Add clusters for clear reasons, and manage them as one fleet from Git.
5. **Budget for day 2 from day 1.** Kubernetes ships about three releases a year. Upgrades, security patches and on-call support are permanent work, and people usually cost more than the servers.

02

Why Kubernetes platforms disappoint

The benefits of Kubernetes are real, but only when the platform is designed for the teams that use it and run by a team that owns it. When we review platforms that have not delivered, the same patterns appear:

What we find	What it leads to
Adopted because everyone else did	Five applications, one team, and a platform that needs more care than the applications it runs.
Clusters everywhere	Every team built its own cluster its own way. Nobody knows which ones are patched or who owns them.
Upgrades nobody dares to do	Clusters fall three or four versions behind, lose security fixes, and an upgrade becomes a project instead of a routine.
Developers still wait for tickets	The platform exists, but a namespace, a database or a DNS name still needs a request to the infrastructure team.
Security left at defaults	Containers run as root, any pod can reach any other, images are never scanned and secrets sit in Git.
Too many tools at once	Service mesh, three monitoring stacks and a portal in month one, before a single team is in production.
No backup of the platform itself	Applications are backed up, but cluster configuration, secrets and persistent volumes are not.

> **In short:** most failed platforms were either not needed, or needed and never owned. The fix is a fit test at the start and a platform team with a clear mandate after it.

03

What a platform actually is

Kubernetes schedules containers across a pool of servers, restarts them when they fail, rolls out new versions gradually and scales them with load. It does this from declared desired state: you describe what should run, and controllers keep reality matching that description. That is why Git-based delivery fits it so well.

A cluster on its own is not a platform. A developer needs a path from code to a running, monitored, secure service without knowing how the cluster works. That path is made of several layers:

Layer	What it does	Common choices
Clusters	Run the containers	Managed (EKS, AKS, GKE, OKE) or self-run (RKE2, OpenShift, Kubernetes on OpenStack, kubeadm)
Delivery	Get code from Git to production safely	CI pipelines, Argo CD or Flux, Helm or Kustomize
Images	Store, scan and sign container images	Harbor, Trivy, cosign
Networking	Route traffic in and between services	Gateway API or ingress controller, Cilium or Calico
Security	Limit what can run and what it can reach	RBAC, network policies, Kyverno or OPA Gatekeeper, Vault
Observability	Metrics, logs and traces per team	Prometheus, Grafana, Loki, OpenTelemetry
Developer experience	Self-service templates and documentation	Backstage or a simple internal portal

The platform as a product

Platform engineering means running these layers as an internal product. The platform team has customers (the development teams), a backlog, release notes and service levels. Its success is measured by what developers can do without asking: how long it takes a new service to reach production, how often teams deploy, how many deployments fail, and how quickly they recover. Those four measures are the DORA metrics, and they are a better test of a platform than the number of clusters or tools.

Golden paths

A golden path is a supported, documented way to do a common job, such as “a Java API with a PostgreSQL database” or “a scheduled batch job”. One template creates the repository, pipeline, deployment configuration, namespace, dashboards and alerts, with security settings already correct. Teams can leave the path, but the easy way and the right way are the same. Two or three good paths cover most of the work in a typical enterprise.

04

The main options compared

There are four realistic ways to run containerised applications. The right choice depends on how many services and teams you have, where they must run, and how much platform work you are willing to own.

Option	Best for	Effort to run	Watch out for
Virtual machines with automation (Ansible, Terraform, Packer)	A handful of applications, stable release pace, existing VM skills	Low to moderate	Slower environments and scaling; drift if automation is not enforced
Managed container service or PaaS (ECS, Azure Container Apps, Cloud Run)	Small teams in one public cloud who want containers without a cluster	Low	Provider-specific features; limits on networking and tenancy
Managed Kubernetes (EKS, AKS, GKE, OKE)	Many services and teams in public cloud	Moderate: provider runs the control plane, you run the rest	Cloud cost; you still own add-ons, upgrades of nodes and add-ons, and security
Self-run Kubernetes (RKE2, OpenShift, Kubernetes on OpenStack or bare metal)	On-premises, private cloud, strict data rules, GPUs, edge sites	High: you run everything, including the control plane and etcd	Upgrade discipline, on-call ownership, hardware replacement

Managed or self-run

In public cloud, managed Kubernetes is almost always the right start. The provider runs and upgrades the control plane, and the platform team concentrates on what developers see. On-premises, in a private cloud or where data must stay in a specific place, you run Kubernetes yourself. Choose a well-supported distribution with a documented upgrade path, and plan for the people to run it before you buy hardware.

Where Kubernetes runs on-premises

- **On OpenStack:** clusters built by Cluster API or Magnum, with Cinder for volumes and Octavia for load balancers. Good when tenants of an existing private cloud want clusters on demand.
- **On VMware or KVM virtual machines:** the simplest on-premises start, using familiar operations, with some overhead from the virtualisation layer.
- **On bare metal:** the best performance and direct GPU access. Needs MetalLB or similar for load balancers and a plan for replacing failed servers.

05

Do you need Kubernetes? A fit test

Ask these five questions before committing. They take an afternoon and save months.

1. **How many services, and how many teams?** Below roughly ten services and two teams, the platform overhead usually outweighs the benefit. Above thirty services or five teams, a shared platform starts to pay for itself.
2. **How often do you release?** If most applications change monthly or less, automated VMs are enough. If teams want to deploy daily, independently, Kubernetes and GitOps help a great deal.
3. **Does load vary sharply?** Festive sales, results days, month-end runs and campaign launches are where automatic scaling earns its keep.
4. **Where must it run?** Public cloud only points to managed Kubernetes or a container service. On-premises, several sites or edge locations point to a self-run distribution with fleet management.
5. **Who will run it?** If you cannot name at least two people who will own the platform, upgrade it and carry the pager, start with something simpler or a managed service.

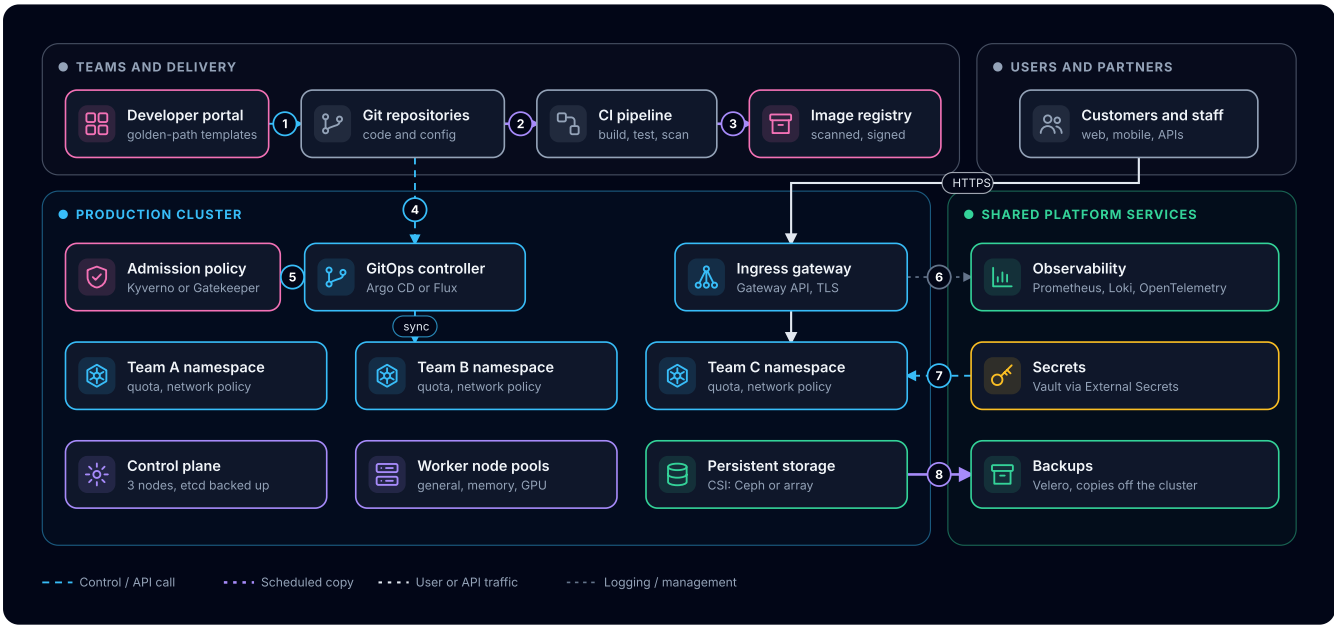
Your situation	Usual answer
Under 10 services, 1 or 2 teams, monthly releases	Automated VMs or a managed container service
10 to 30 services, a few teams, public cloud	Managed Kubernetes, one production cluster, lean add-ons
30+ services, many teams, on-premises or regulated data	Self-run distribution, a platform team, golden paths
Many edge or branch sites	Small standard clusters managed centrally from Git
GPU training and inference	Kubernetes with GPU node pools, or a dedicated scheduler for large training jobs

Not every workload belongs on the platform. Large commercial databases, licence-bound software and appliances are often better on VMs or managed services even when everything else moves. Decide per workload, not per organisation.

06

Reference architecture

The diagram shows a lean production platform. Developers never change the cluster directly. Code goes through CI, images through the registry, and everything the cluster runs, including policies and add-ons, comes from Git.



Numbered flows: (1) a team creates a new service from a portal template, which sets up the repository and configuration, (2) CI builds and tests on every merge, (3) images are scanned, signed and pushed to the private registry, (4) the GitOps controller watches Git, (5) every change passes admission policy before it is accepted, (6) traffic, metrics, logs and traces go to shared observability, (7) secrets come from Vault at run time, never from Git, (8) cluster resources and volumes are backed up off the cluster.

Cluster design

- **Control plane:** three nodes, so it survives one failure and can be upgraded one node at a time. Back up etcd on a schedule and test the restore.

- **Node pools:** separate pools for general workloads, memory-heavy workloads and GPUs, with labels and taints so work lands in the right place.
- **Networking:** a CNI that enforces network policies (Cilium or Calico), Gateway API or an ingress controller for inbound traffic, and an agreed IP plan that does not clash with the corporate network.
- **Storage:** a CSI driver for your storage, such as Ceph via Rook, a vendor array or cloud volumes, with storage classes per performance tier.

GitOps delivery

With GitOps, Git is the single source of truth for what runs. A controller such as Argo CD or Flux watches Git and keeps the cluster matching it. Deployments become pull requests and rollbacks become reverts, and every change is reviewed and recorded.

- Keep application code and deployment configuration separate, in different repositories or folders.
- Promote between environments through Git, using the same image, never by rebuilding.
- Turn on automatic drift correction and pruning in non-production first, then in production once teams trust it.
- Use progressive delivery (Argo Rollouts or Flagger) for services where a bad release is expensive.

Tenancy and multiple clusters

Inside a cluster, give each team and environment a namespace with resource quotas, default-deny network policies and role bindings from your identity provider, all created from a template. Where tenants need stronger isolation, for example separate customers on a SaaS platform, virtual clusters (vCluster) or separate node pools add a further boundary. Add a whole cluster only for a clear reason:

Reason for another cluster	Usually worth it?
Separate production from non-production	Yes
Data must stay in a specific location	Yes
Users in two regions, or disaster recovery	Yes
Edge or plant sites that must run when the WAN is down	Yes, as small standard clusters
Each team wants its own	No: use namespaces, quotas and policies
One application cannot upgrade yet	No: fix the application; old clusters become a risk

When there are several clusters, manage them as one fleet: build every cluster from the same code (Cluster API, Terraform or your distribution’s tooling), apply the same baseline add-ons and policies from Git, for example with Argo CD ApplicationSets, send all telemetry to one place, and upgrade in waves.

07

Sizing and cost

Worker capacity

Size from what pods request at peak, not from what servers cost. A first estimate:

```
Worker nodes = (peak pod requests + platform add-ons) ÷ target utilisation ÷ allocatable per node, rounded up, + 1 spare node
```

Work it out for CPU and memory separately and take the larger answer. The spare node lets the cluster survive one node failure and still have room for rolling updates. Target utilisation of 65 to 75 percent leaves headroom for spikes and for the scheduler to place pods.

A worked example

A retailer runs 30 services. At festive peak they average 4 replicas each, so 120 pods, each requesting 0.5 vCPU and 1 GiB of memory. Platform add-ons (ingress, GitOps, policy, monitoring, logging agents) need about 10 vCPU and 24 GiB.

Step	CPU	Memory
Application pods at peak	120 × 0.5 = 60 vCPU	120 × 1 = 120 GiB
Plus platform add-ons	60 + 10 = 70 vCPU	120 + 24 = 144 GiB
At 70% target utilisation	70 ÷ 0.7 = 100 vCPU	144 ÷ 0.7 ≈ 206 GiB
Nodes of 16 vCPU, 64 GiB (about 15 vCPU, 58 GiB allocatable)	100 ÷ 15 = 6.7, so 7	206 ÷ 58 = 3.6, so 4
Production workers (larger figure + 1 spare)	7 + 1 = 8 nodes	CPU sets the size

Add three small control plane nodes if self-run, and a non-production cluster at roughly half the size. Outside the festive season, the cluster autoscaler can drop production to five or six workers, which is where the efficiency gain shows up.

What it costs to run, per year

Item	Planning range (managed Kubernetes, Indian cloud region)
Production workers: 8 nodes at about ₹45,000 to ₹60,000 per month each	₹43 to ₹58 lakh
Non-production: 4 nodes of the same size	₹22 to ₹29 lakh
Control plane fees, load balancers, storage, log retention	₹12 to ₹22 lakh
Optional support subscriptions for distribution or tools	₹0 to ₹40 lakh
Platform team: 3 engineers, fully loaded	₹75 lakh to ₹1.2 crore
Total	**About ₹1.5 to ₹2.5 crore**

These are planning ranges, not quotes. Reserved or committed-use pricing usually cuts compute by 30 to 40 percent; on-premises hardware moves the cost into capital spend over three to five years. Notice

that people are the largest single line. That is normal, and it is why the fit test asks who will run the platform before anything else.

> **Does Kubernetes save money?** It can, by packing workloads more tightly and scaling down when idle. But it adds platform and people cost. The bigger gains are usually faster, safer releases and less time spent on environments, so measure those too.

08

Security and governance

Out of the box, Kubernetes is permissive: containers can run as root and any pod can reach any other. A small set of controls, applied by default from templates and enforced at admission, closes most of the gaps. Keeping clusters upgraded closes most of the rest.

Control	What it does	Common choices
Access control	People and pipelines get only the permissions they need	RBAC bound to identity provider groups via OIDC, no shared admin accounts
Namespaces and quotas	Teams are separated and cannot use up the cluster	One namespace per team and environment, from a template
Network policies	Pods only talk to what they need to	Default deny, then allow, with Cilium or Calico
Pod security	Blocks root containers and risky settings	Pod Security Standards at the restricted level
Policy engine	Enforces your rules whenever anything is created	Kyverno or OPA Gatekeeper
Secrets	Keeps passwords and keys out of Git and images	Vault or a cloud secret store via External Secrets
Audit and runtime	Records changes and spots unusual behaviour	API audit logs to your SIEM, Falco
Benchmarks	Checks clusters against known good settings	CIS Kubernetes Benchmark with kube-bench

Software supply chain

Attackers increasingly target the delivery pipeline rather than the running system. Protect the path from code to cluster:

- Build images only in CI, from approved base images, and generate a software bill of materials (SBOM) for each.
- Scan images for known vulnerabilities on build and again on a schedule, because new CVEs appear after release.
- Sign images with cosign and have admission policy reject anything unsigned or from an unknown registry.
- Protect the CI system and Git itself: branch protection, required reviews, and short-lived credentials for pipelines.

Governance without tickets

Write policies as code, test them in CI like any other change, and run new policies in audit mode for a few weeks before enforcing them. Map controls to your obligations, such as CERT-In 6-hour incident reporting, which needs audit logs you can search quickly, and the DPDP Act, which needs you to know where personal data lives. Sector regulators such as RBI, SEBI and IRDAI expect evidence of access control, change records and patching; a GitOps platform produces most of that evidence as a side effect.

09

Operating it: day 2

Observability

Each team should see its own metrics, logs and alerts, so problems are found by the people who can fix them. Collect metrics with Prometheus, logs with Loki or your existing log platform, and traces with OpenTelemetry, and give every golden-path service a standard dashboard. Alert on what users feel, error rate and latency against a service level objective, not on every CPU spike. The platform team watches the platform itself: control plane health, etcd latency, node pressure, certificate expiry and GitOps sync failures.

Upgrades as a routine

Kubernetes ships about three minor versions a year, and each is supported upstream for roughly fourteen months. Managed services follow a similar window. A cluster more than two versions behind is hard to upgrade and is losing security fixes.

- Upgrade on a calendar, every four to six months: non-production first, a short soak, then production.
- Check manifests for deprecated APIs before each upgrade, with tools such as Pluto or kubent, so applications do not break.
- Upgrade add-ons (CNI, ingress, GitOps, policy engine) on the same rhythm; they drift faster than the cluster.

Backup and recovery

Because cluster state lives in Git, most of a cluster can be rebuilt from code. What Git does not hold must be backed up: etcd snapshots, persistent volumes, and secrets that are not in an external store. Velero handles cluster resources and volumes, with copies kept off the cluster and ideally immutable. Rebuild a non-production cluster from scratch at least twice a year to prove it works.

The team model

A small platform on managed Kubernetes can be run by two or three engineers with good automation. A self-run, multi-cluster platform needs a dedicated team of four to eight, with on-call cover. Keep the split clear: the platform team owns clusters, add-ons, golden paths and upgrades; application teams own their services, their alerts and their on-call. A platform team that also runs everyone’s applications becomes a bottleneck, which is exactly what platform engineering is meant to remove.

10

A practical roadmap

A first platform does not need to be big. It needs to be used. The pace is set by how quickly the first teams move real services, not by building the cluster.

Stage	Typical duration	Outcome
1. Fit check	2 to 3 weeks	Applications, teams, release pace and skills reviewed. A clear yes, no, or not yet, with the option chosen.
2. Lean first platform	6 to 10 weeks	One production and one non-production cluster, GitOps delivery, baseline security, monitoring, one golden path.
3. First teams on board	6 to 8 weeks	Two or three willing teams move real services and help shape the templates and documentation.
4. Harden	4 to 6 weeks	Policies enforced, image signing, backups tested, first planned upgrade done, on-call in place.
5. Scale out	Ongoing, by quarter	More teams and golden paths, self-service portal, extra clusters only where a reason exists.
6. Hand over	4 weeks, overlapping	Runbooks, upgrade practice and platform backlog owned by your platform team.

11

Ten common mistakes

1. **Adopting Kubernetes for a handful of applications.** The platform becomes the biggest system you run.
2. **Building the cluster and calling it a platform.** Developers still need delivery, templates and monitoring.
3. **One cluster per team.** Patching and upgrading multiply; namespaces would have done the job.
4. **Falling behind on versions.** Each skipped upgrade makes the next one harder and riskier.
5. **Changing clusters by hand.** Changes made outside Git are lost or quietly overwritten.
6. **Leaving security at defaults.** Root containers, open east-west traffic and unscanned images.
7. **Secrets in Git or in images.** They are copied everywhere and almost impossible to rotate.
8. **Too many tools in the first months.** A service mesh and three dashboards before the first team goes live.
9. **No resource requests or limits.** One noisy service starves the rest and capacity planning becomes guesswork.
10. **No named platform owner.** Without a team that owns upgrades and on-call, the platform slowly decays.

12

Platform readiness checklist (36 points)

Use this list to score a planned platform or one you already run. Anything you cannot tick with evidence is a gap worth closing.

Fit and ownership

- [ ] Fit test done, with service count, team count and release pace written down
- [ ] Option chosen (VMs, container service, managed or self-run Kubernetes) with the reason recorded
- [ ] Named platform owner and at least two engineers who carry the pager
- [ ] Platform success measures agreed, such as lead time and deployment frequency
- [ ] Workloads that stay off the platform listed, with reasons
- [ ] Running cost, including people, approved for three years

Cluster design

- [ ] Production and non-production on separate clusters
- [ ] Three control plane nodes, with etcd backups restored in a test
- [ ] Worker capacity sized from peak requests, with one spare node
- [ ] Separate node pools for general, memory-heavy and GPU workloads where needed
- [ ] Pod and service IP ranges agreed with the network team
- [ ] Storage classes defined per performance tier through a CSI driver

Delivery and GitOps

- [ ] Everything the cluster runs, including add-ons and policies, comes from Git
- [ ] Application code and deployment configuration kept separate
- [ ] Promotion between environments through Git, using the same image
- [ ] Rollback by reverting a commit, practised at least once
- [ ] Drift correction turned on, starting with non-production
- [ ] Every cluster built from the same code

Security

- [ ] RBAC bound to identity provider groups, with no shared admin accounts
- [ ] Default-deny network policies in every namespace
- [ ] Pod Security Standards enforced at the restricted level
- [ ] Images scanned on build and on a schedule, and signed
- [ ] Admission policy rejects unsigned images and unknown registries
- [ ] Secrets held in Vault or a cloud store, never in Git
- [ ] API audit logs sent to the SIEM and searchable within hours

Operations and day 2

- [ ] Each team sees its own dashboards, logs and alerts
- [ ] Alerts based on service level objectives, not raw resource use
- [ ] Upgrade calendar set, with no cluster more than two versions behind
- [ ] Deprecated APIs checked before every upgrade
- [ ] Velero backups kept off the cluster and restored in a test
- [ ] A non-production cluster rebuilt from code twice a year

Developer experience

- [ ] At least one golden path from template to production
- [ ] A new service reaches production without a ticket
- [ ] Platform documentation kept next to the templates and up to date
- [ ] Resource requests and limits set by default in templates
- [ ] Platform backlog reviewed with application teams every quarter

13

Glossary

Term	Meaning
Pod	The smallest unit Kubernetes runs: one or more containers that share network and storage.
Namespace	A named section of a cluster used to separate teams or environments, with its own quotas and permissions.
Control plane and etcd	The components that manage the cluster; etcd is the database that holds all cluster state.
GitOps	Running systems from declared state in Git, with a controller that keeps the cluster matching it.
Admission policy	Rules checked when anything is created or changed in the cluster, such as rejecting unsigned images.
CNI and CSI	Plug-in interfaces for container networking and for storage.
Golden path	A supported, documented template for a common job, from repository to production.
SBOM	Software bill of materials: a list of the components and versions inside an image.
DORA metrics	Lead time for changes, deployment frequency, change failure rate and time to restore service.

About Vakratron Systems

Vakratron Systems LLP is a vendor-neutral infrastructure design firm, recognised as a startup by DPIIT. We design data centre, disaster recovery, cloud, GPU and AI platforms for enterprises and government buyers. Every design starts from the workload, compares realistic options with their costs, writes its assumptions down, and is tested before handover.

How we can help with Kubernetes and platform engineering

- **Fit check and platform review:** an honest view in two to three weeks of whether Kubernetes fits, or what to fix in the platform you already run.
- **Lean platform design:** cluster layout, tenancy, GitOps delivery, security baseline, sizing and a cost model, including the people needed to run it.
- **First teams and upgrades:** onboarding two or three teams onto golden paths, and setting up an upgrade routine your team can repeat.

Talk to us
connect@vakratronsys.com

Online
vakratronsys.com/advisory

This paper gives general guidance based on common enterprise requirements. Figures are planning ranges, not quotations. Apply them to your environment with care, or ask us to help. © 2026 Vakratron Systems LLP.